

第6章: プロジェクトのセットアップ (ナビゲーション)

ここから実際に「書籍管理アプリ」を組み立てていきます。この章では、アプリ全体の画面構成（ナビゲーション）を作り、本の「型」を定義し、Supabaseとデータをやり取りする関数をまとめます。第7・8章のCRUD実装の土台になる、大切な準備の章です。

1. この章で作るもの（全体像）

これから作るファイルと、その役割を先に示します。

```
my-books-app/  
├── app/  
│   ├── _layout.tsx      ← アプリ全体の枠組み（Stackナビゲーション）  
│   ├── index.tsx        ← 書籍一覧画面（第7章で中身を作る）  
│   ├── new.tsx          ← 新規登録画面（第7章）  
│   └── books/  
│       └── [id].tsx      ← 編集画面（第8章。idで対象の本を識別）  
├── lib/  
│   ├── supabase.ts      ← Supabase接続（第5章で作成済み）  
│   └── books.ts          ← 本のCRUD関数をまとめる（この章で作る）  
└── types/  
    └── book.ts           ← 本の「型（Book型）」を定義（この章で作る）
```

本書のナビゲーション方針: 第4章で「Tabs（下タブ）」にも触れましたが、書籍管理アプリはシンプルに「一覧 → 登録／編集」と画面を重ねる **Stack ナビゲーション** で作ります。Expoテンプレートに最初から入っている **(tabs)** フォルダは使わない構成に整理します。

2. 本の「型」を定義する

第2章で学んだ **type** を使い、アプリ全体で使う「本のデータの形」を1か所に定義します。こうすると、どのファイルでも同じ型を使えて安全です。

プロジェクト内に **types** フォルダを作り、**book.ts** を新規作成します。

▼このコードがやること（先に日本語で）：アプリ全体で使う「本のデータの形（型）」を1か所にまとめて定義します。**Book** は「すでに保存済みの本」を表す型、**NewBook** は「これから登録する本」を表す型で、IDや登録日時の有無で2つに分けています。こうして型を1か所に置いておくと、どの画面でも同じ形を使い回せて、入力ミスエディタが早めに教えてくれます（各項目の意味はコード内のコメントで説明します）。

```
// types/book.ts - アプリ全体で使う「本」の型を定義する

// Book : 1冊の本を表す型。Supabaseのbooksテーブルの列と対応させる（第5章で作った表）
export type Book = {
  id: string;           // 本のID (Supabaseが自動生成するuuid)
  title: string;        // タイトル (必須)
  author: string;       // 著者 (必須)
  status: string;       // 読書状態 ("未読" / "読書中" / "読了")
  memo: string | null;  // メモ (空のこともあるので string または null)。第2章の「| null」参照
  created_at: string;   // 登録日時 (Supabaseが自動で入れる)
};

// NewBook : 「新規登録するときの入力データ」の型
// id や created_at はSupabaseが自動で付けるので、入力時には不要。だからそれらを除いた形にする
export type NewBook = {
  title: string;
  author: string;
  status: string;
  memo: string | null;
};
```

string | null の意味（復習）：|（パイプ 縦棒）は「または」を表し、「string 型 または null」という意味です。これを「ユニオン型（union type）」と呼びます。memoは入力されないこともあるので、文字列かnullのどちらかを許す型にしています。

なぜ Book と NewBook を分ける？ 既存の本（Book）はIDや登録日時を持っていますが、これから新しく作る本（NewBook）にはまだIDがありません（Supabaseが保存時に発番するため）。入力時に余計な項目を要求しないよう、型を分けておくと安全で分かりやすくなります。

3. CRUD関数をまとめる — lib/books.ts

Supabaseとのデータのやり取り（取得・追加・更新・削除）を、各画面に散らばらず1つのファイルにまとめます。こうすると、画面側のコードがすっきりし、後から修正も楽になります。

lib フォルダに **books.ts** を新規作成します。第7・8章で各関数を実際に使います。

▼このコードがやること（先に日本語で）：Supabase（データの保管庫）と本のデータをやり取りする関数を、まとめて1つのファイルに用意します。「全件取得」「1件追加」「1件取得」「更新」「削除」という、いわゆる **CRUD**（**作成・読取・更新・削除**）の5つの関数を作ります。ポイントは、画面のコードからは **fetchBooks()** のように関数を呼ぶだけで済むようにし、データベースとのやり取りの細かい書き方をこのファイルに閉じ込めることです（各関数の中身はコード内のコメントで説明します）。

```
// lib/books.ts - 本のCRUD（作成・読取・更新・削除）をまとめたファイル

import { supabase } from "../supabase"; // 第5章で作った接続オブジェクトを借りる
import { Book, NewBook } from "../types/book"; // さっき作った型を借りる

// =====
// Read（読み取り）：本を全件取得する
// =====
// async：中で await（待つ処理）を使うので付ける（第2章参照）
// : Promise<Book[]>：この関数は「最終的にBookの配列を返す」という戻り値の型
export async function fetchBooks(): Promise<Book[]> {
  // supabase.from("books")：booksテーブルを対象にする
  // .select("*")：全列(*)を取得する
  // .order("created_at", { ascending: false })：created_atで並べ替え。降順（新しい順）
  const { data, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });

  if (error) throw error; // エラーがあれば throw で呼び出し元に知らせる（第7章で受け止める）
  return data ?? []; // データを返す。?? は「左がnull/undefinedなら右を使う」。空なら[]
  // を返す
}

// =====
// Create（作成）：本を1冊追加する
// =====
// 引数 newBook：登録する本のデータ（NewBook型）
export async function createBook(newBook: NewBook): Promise<void> {
  // .insert(newBook)：受け取ったデータをbooksテーブルに追加する
  const { error } = await supabase.from("books").insert(newBook);
  if (error) throw error;
}

// =====
// Read（1件）：idを指定して1冊だけ取得する（編集画面で使う）
// =====
export async function fetchBookById(id: string): Promise<Book> {
  const { data, error } = await supabase
    .from("books")
    .select("*")
    .eq("id", id) // .eq("id", id)：id列が引数idと等しい行に絞り込む（eq = equal）
    .single(); // .single()：結果を「1件のオブジェクト」として受け取る（配列でな
    <）

  if (error) throw error;
  return data;
}

```

```
// =====
// Update (更新) : idの本を新しい内容に書き換える
// =====
export async function updateBook(id: string, updated: NewBook): Promise<void> {
  const { error } = await supabase
    .from("books")
    .update(updated) // .update(updated) : 指定データで上書きする
    .eq("id", id);   // .eq("id", id) : id列が一致する行だけを更新 (これが無いと全件更新
                     // になり危険)
  if (error) throw error;
}

// =====
// Delete (削除) : idの本を削除する
// =====
export async function deleteBook(id: string): Promise<void> {
  const { error } = await supabase
    .from("books")
    .delete()        // .delete() : 行を削除する
    .eq("id", id);   // .eq("id", id) : id列が一致する行だけを削除
  if (error) throw error;
}
```

Supabaseのメソッドチェーン: `supabase.from("books").select("*").order(...)` のように、`.` でつないで条件を足していく書き方を「メソッドチェーン」と呼びます。「booksテーブルから → 全列を選んで → 並べ替える」と、左から右へ読めば処理が分かります。

throw error の意味: `throw` (スロー) は「エラーを呼び出し元に投げて知らせる」命令です。ここでエラーを投げおくと、画面側で `try / catch` (第7章で説明) を使って「エラーなら画面にメッセージを出す」といった対応ができます。

?? [] (ヌル合体演算子): `data ?? []` は「`data` が `null` か `undefined` なら、代わりに空配列 `[]` を使う」という意味です。「データが取れなかったときも、せめて空のリストを返す」ことで、画面側がエラーになりにくくなります。

4. ナビゲーションの枠組みを作る — `_layout.tsx`

アプリ全体の画面の重ね方 (Stack) と、各画面のヘッダー (上部バー) のタイトルを設定します。 `app/_layout.tsx` を次の内容にします (既存内容は置き換え)。

▼このコードがやること（先に日本語で）：アプリ全体の「画面の重ね方」と「上部バー（ヘッダー）の見た目」をまとめて設定します。**Stack** は画面をカードのように積み重ねて遷移させる仕組みで、共通の設定（色など）は全画面まとめて、タイトルは画面ごとに指定します。**_layout.tsx** はアプリの土台になる特別なファイルで、ここで一度決めておけば各画面に同じ枠組みが自動で適用されます（細かい指定はコード内のコメントで説明します）。

```
// app/_layout.tsx - アプリ全体のナビゲーションの枠組み

import { Stack } from "expo-router"; // Stack（カードを重ねる遷移）を借りる

export default function RootLayout() {
  return (
    // Stack : 画面を積み重ねるナビゲーション。screenOptions で全画面共通の見た目を指定
    <Stack
      screenOptions={{
        headerStyle: { backgroundColor: "#1e40af" }, // ヘッダー（上部バー）の背景色を青に
        headerTintColor: "#fff", // ヘッダーの文字・戻る矢印の色を白に
        headerTitleStyle: { fontWeight: "bold" }, // ヘッダータイトルを太字に
      }}
    >
      {/* Stack.Screen : 各画面ごとの個別設定。name はファイル名（拡張子なし）と対応 */}
      <Stack.Screen name="index" options={{ title: "📖 書籍管理" }} />
      {/* index.tsx → タイトル「書籍管理」。これが最初に表示される画面 */}

      <Stack.Screen name="new" options={{ title: "新規登録" }} />
      {/* new.tsx → タイトル「新規登録' */}

      <Stack.Screen name="books/[id]" options={{ title: "編集" }} />
      {/* books/[id].tsx → タイトル「編集」。[id]は動的な値（第4章参照） */}
    </Stack>
  );
}
```

screenOptions と **options** の違い: - **screenOptions**（Stack全体に付ける）... **すべての画面に共通**する設定。 - **options**（各 **Stack.Screen** に付ける）... その**画面だけ**の設定（タイトルなど）。こうして「色は全画面共通、タイトルは画面ごと」と整理できます。

name はファイル名と一致させる: `<Stack.Screen name="books/[id]" ... />` の **name** は、`app/` フォルダからのファイルのパス（拡張子なし）と一致させます。`app/books/[id].tsx` なら `name="books/[id]"` です。一致していないと設定が効きません。

4.1 不要なテンプレートファイルの整理

Expoのテンプレートには `app/(tabs)/` フォルダなどサンプル画面が入っています。本書のStack構成に合わせ、次のように整理します。

```
# ターミナルで、不要なテンプレート画面フォルダを削除する例（プロジェクト直下で実行）
# 注意： 削除は慎重に。中身を確認してから実行してください
# rm はファイル/フォルダを削除するコマンド / -r は「フォルダごと（再帰的に）」の意味
rm -r app/(tabs)
# Windowsの PowerShell では次のように書く：
# Remove-Item -Recurse -Force "app/(tabs)"
```

削除に不安があれば、無理に消さなくても進められます。その場合は、これから作る `index.tsx` などを `app/` 直下に置けば、そちらが優先して使われます。慣れないうちは「新しいファイルを `app/` 直下に作る」方針でも構いません。整理は後からでもできます。

5. プレースホルダー画面を置いて動作確認

第7・8章で中身を作る前に、まず「空っぽの画面」を置いて、ナビゲーション（画面遷移）が動くことを確認します。

5.1 一覧画面（仮）

`app/index.tsx` を次の内容にします。

▼ このコードがやること（先に日本語で）：最初に表示される「書籍一覧画面」を、まずは中身が空の仮の状態で作ります（本格的な一覧表示は第7章で実装します）。今回の目的は、画面に置いたボタンを押すと新規登録画面へ移動できることを確かめることです。`useRouter` で画面移動の道具を取り出し、`router.push("/new")` で別の画面へ進みます（各部分の意味はコード内のコメントで説明します）。

```
// app/index.tsx - 書籍一覧画面（この章では仮の内容。第7章で本実装）

import { View, Text, Pressable, StyleSheet } from "react-native";
import { useRouter } from "expo-router"; // 画面移動の道具（第4章参照）

export default function HomeScreen() {
  const router = useRouter(); // 移動操作を取得

  return (
    <View style={styles.container}>
      <Text style={styles.text}>ここに書籍一覧が表示されます（第7章で実装）</Text>

      {/* 新規登録画面へ移動するボタン */}
      <Pressable style={styles.button} onPress={() => router.push("/new")}>
        {/* router.push("/new") : new.tsx（新規登録画面）へ移動 */}
        <Text style={styles.buttonText}>+ 新規登録へ</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", gap: 16, padding: 20 },
  text: { fontSize: 14, color: "#475569", textAlign: "center" },
  button: { backgroundColor: "#1e40af", paddingVertical: 12, paddingHorizontal: 20, borderRadius: 10 },
  buttonText: { color: "fff", fontWeight: "bold" },
});
```

5.2 新規登録画面（仮）

`app/new.tsx` を新規作成します。

▼ このコードがやること（先に日本語で）：「新規登録画面」を、こちらまずは中身が空の仮の状態で作ります（登録フォームの本体は第7章で実装します）。狙いは、一覧画面から移動してきたあと、ボタンで元の画面（一覧）へ戻れることを確かめることです。`router.back()` は「1つ前の画面に戻る」操作で、これでStackナビゲーションの行き来が一通り確認できます（各部分の意味はコード内のコメントで説明します）。


```
// app/new.tsx - 新規登録画面（この章では仮の内容。第7章で本実装）

import { View, Text, Pressable, StyleSheet } from "react-native";
import { useRouter } from "expo-router";

export default function NewBookScreen() {
  const router = useRouter();

  return (
    <View style={styles.container}>
      <Text style={styles.text}>ここに登録フォームが表示されます（第7章で実装）</Text>

      { /* 前の画面（一覧）へ戻るボタン */ }
      <Pressable style={styles.button} onPress={() => router.back()}>
        { /* router.back() : 1つ前の画面に戻る */ }
        <Text style={styles.buttonText}>戻る</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", gap: 16, padding: 20 },
  text: { fontSize: 14, color: "#475569", textAlign: "center" },
  button: { backgroundColor: "#64748b", paddingVertical: 12, paddingHorizontal: 20, borderRadius: 10 },
  buttonText: { color: "fff", fontWeight: "bold" },
});
```

5.3 動作確認

`npx expo start` でアプリを起動し、スマホ（Expo Go）で確認します。

1. 最初に「書籍管理」というタイトルの一覧画面（仮）が表示される。
2. 「+ 新規登録へ」を押すと、「新規登録」画面にスライドして遷移する。
3. ヘッダー左の「戻る矢印」または「戻る」ボタンで一覧に戻る。

これが確認できれば、ナビゲーションの土台は完成です。Stackナビゲーションが「カードを重ねて・めくる」動きをしているのが分かるはずです。

うまく動かないときは: - ヘッダーが青くならない → `_layout.tsx` の保存を確認。 - 「Unmatched Route」と出る → ファイル名（`new.tsx` など）とパス（`/new`）が一致しているか確認。 - 画面が真っ白 → ターミナルのエラーを確認。多くは `import` のパスミス。

6. この章のまとめ

- アプリ全体で使う `Book` / `NewBook` 型を `types/book.ts` に定義した
- Supabaseとのやり取り（`fetchBooks` / `createBook` / `fetchBookById` / `updateBook` / `deleteBook`）を `lib/books.ts` に集約した
- `_layout.tsx` で Stack ナビゲーションとヘッダーの見た目を設定した
- 仮の 一覧画面・新規登録画面 を置き、画面遷移（`push` / `back`）が動くことを確認した

次の章へ: 土台ができました。第7章では、この `lib/books.ts` の関数を使って、実際に本の一覧を表示し、新しい本を登録する機能（CRUDのReadとCreate）を実装します。